BUILD PLAN

I am building an MVP for the Alloy River AI Product Builder assignment: a personal Timex watch scout for one known collector.

The product helps a vintage Timex collector stay on top of interesting listings across marketplaces without repeatedly checking each site manually. The user's preferences are already known from the assignment prompt, so this is not a generic preference-input tool. It is a focused scout that uses a stored collector profile, recent purchase examples, deterministic validation, and LLM-assisted recommendation reasoning to surface the best current candidates.

USER PROBLEM

"I collect vintage Timex watches and want a better way to stay on top of interesting listings across the market. Today, finding good pieces means manually checking multiple sites, remembering what I like, and hoping I don't miss something.

"Build me a tool that:

- *Syncs active listings from key marketplaces (eBay, Chrono24, and any others you think are worth including).*
- *Lets me surface listings that match my preferences.*
    - *<$50 in total cost (including shipping to M6K1V8)*
    - *Isn't explicitly broken – I'm willing to replace a battery.*
    - *Is interesting. Collabs, deadstock, vintage models,*
      3 watches I've bought recently:
        - *https://www.ebay.ca/itm/377073705816*
        - *https://www.ebay.ca/itm/117111976291*
        - *https://www.etsy.com/ca/listing/4469739360*
- *Highlights and identifies potential purchase candidates worth paying attention to."*

ROLE CONTEXT

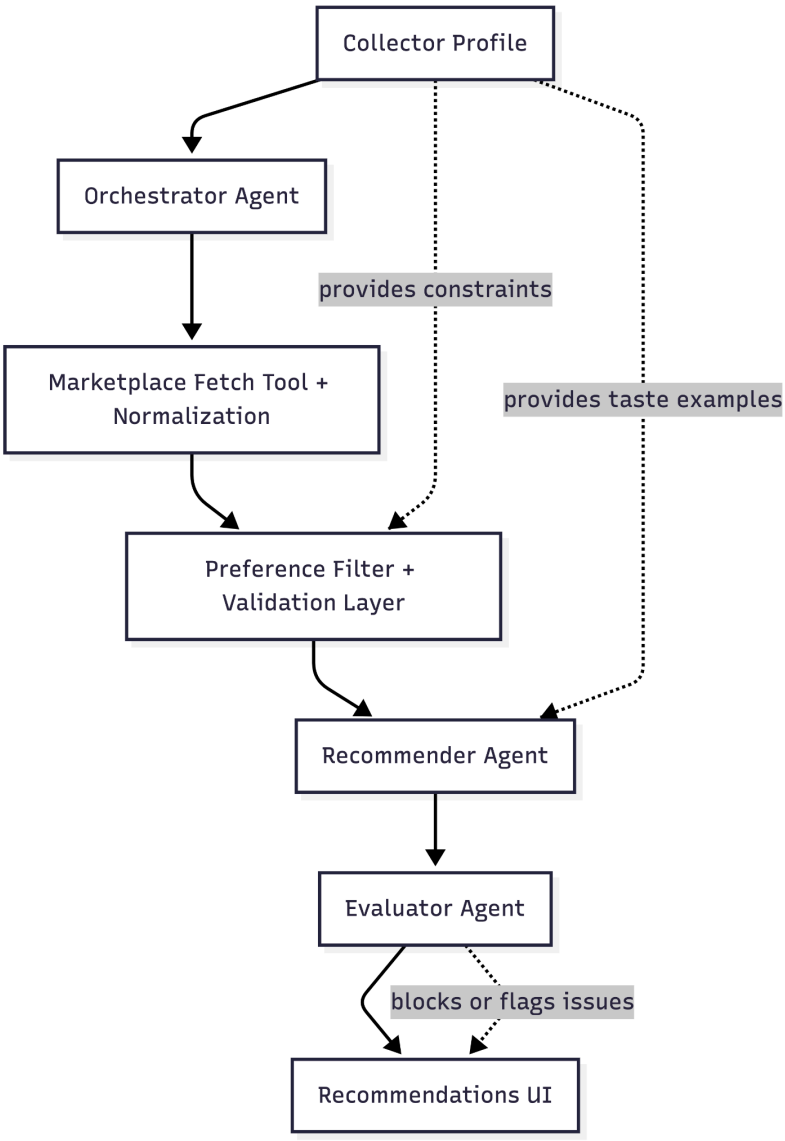
The assignment also maps to three important expectations from the role.

- Deep customer empathy: translate a real user's stated preferences into something buildable.
- AI fluency: use agents where judgment helps, and use deterministic code where correctness matters more.
- Financial and transactional accuracy: handle money carefully, especially total landed cost, because price plus shipping determines whether a listing is eligible.

SOLUTION SUMMARY

The MVP loads a collector profile, fetches or loads marketplace listings, normalizes them into a shared schema, validates hard constraints, recommends the top three matches, evaluates the recommendations, and displays the results in a thin local web app.

My chosen architecture is shown in this diagram:



The system is scoped around one known collector rather than a broad marketplace app. That keeps the MVP focused and makes the recommendation logic easier to explain: the system is not trying to learn every user's taste; it is applying one collector's known taste to active marketplace candidates.

PROJECT FLOW

1. Load collector profile
2. Fetch listings from marketplace providers or seed data
3. Normalize listings into a common schema
4. Apply deterministic validation and filters
5. Recommend top three candidates with reasoning
6. Run an evaluator check on the recommendations
7. Display recommendations with score, budget check, and risk notes

ARCHITECTURE DECISION TABLE

Component	Agent?	Why?
Light orchestrator	Yes, light agent	Coordinates the workflow, pipeline state
Marketplace fetcher	No, tool/function	Deterministic retrieval and normalization
Filter/validator	Mostly no	Hard constraints should be deterministic
Recommender	Yes	Needs judgment, ranking, explanation
Evaluator	Yes/light agent or LLM check	Reviews recommendation quality and constraint fit

ORCHESTRATOR

The orchestrator is a light agent that coordinates the workflow and passes structured state through the pipeline.

It decides which step runs next, calls the relevant tool or agent, and returns a traceable final result. It should not contain the main intelligence of the product. The meaningful reasoning happens later, in recommendation and evaluation.

FETCHING AND PROVIDER ABSTRACTION

Fetching is a tool, not an agent.

Marketplace retrieval is mostly deterministic: call the provider, collect listing fields, normalize the response, and return structured data. There is no need for an LLM to decide how to fetch records.

The MVP uses a provider abstraction pattern so the system can support live integrations without making the demo depend on them.

Listing provider
├── eBay API provider, key approved
├── Etsy API provider, only if key approved (pending approval)
└── Seed data provider, always works

The seed data provider is part of the product design, not a shortcut around the architecture. API access and third-party approvals can be slow or unreliable, and scraping would make the demo brittle. Seeded listings let the product show the complete user experience reliably: loading candidates, validating constraints, ranking recommendations, and explaining the results.

The seeded listings are realistic and normalized to mirror the fields expected from marketplace APIs, including title, description, price, shipping, marketplace, URL, image URL, seller rating, buying option, condition, and listing date. See `seed_listings.normalized.json`.

When live APIs are available, the seed provider can be swapped out behind the same interface. The rest of the pipeline should not care whether listings came from eBay, Etsy, Chrono24-like data, or seed data.

FETCHING VS FILTERING

Fetching and filtering are separate.

The fetcher retrieves a bounded candidate pool using broad provider-side constraints such as keyword, category, and rough price range. It should not try to encode the full collector taste model.

Filtering happens after normalization, where the system has a consistent schema across marketplaces. This makes the validation logic easier to test and explain.

The system is not designed to ingest entire marketplace inventories for the MVP. It retrieves a practical candidate pool, validates it, and then ranks the best options.

DETERMINISTIC VALIDATION

Filtering is mostly deterministic because the user has hard constraints.

The system should deterministically validate:

- Total cost is under $50 CAD including shipping to M6K1V8
- The item is not explicitly broken
- The listing is active or treated as available in seed data
- Required fields are present
- The marketplace/source is known

The LLM can help only with ambiguous listing language. For example, phrases like “untested,” “needs TLC,” “for parts,” “runs but not checked,” or “battery needed” may require semantic interpretation. The model can flag these as risk signals, but it should not replace hard validation.

FINANCIAL MATH

Financial eligibility is never passed to the LLM.

Price, shipping, currency handling, and total landed cost are calculated deterministically using integer cents. The recommendation agent should only receive listings that already passed the budget validation, plus the computed total cost and any transparent budget notes.

This matters because the user’s budget is a hard constraint, and because the role values careful handling of transactional data. The system should be able to show exactly how it calculated eligibility:

Item price + shipping = total landed cost

THREE REFERENCE PURCHASES AS FEW-SHOT EXAMPLES

The three recently purchased watches are used as few-shot preference examples. See `reference_purchases.normalized.json`.

They are not used as hard filters. Instead, they calibrate the recommender’s sense of taste after deterministic validation has already removed ineligible listings.

The recommender compares eligible candidates against the collector’s demonstrated taste from the reference purchases: vintage Timex character, interesting design, deadstock or unusual models, condition tradeoffs, and overall collector appeal.

This is a good place to use LLM reasoning because “interesting” is subjective and pattern-based. The system should still ground every explanation in source listing data and the collector profile.

RECOMMENDER AGENT

The recommender agent ranks the eligible listings and returns the top three purchase candidates.

Its job is to explain why each listing is worth attention, not to decide whether it satisfies hard constraints. By the time a listing reaches the recommender, budget and obvious condition exclusions have already been checked.

The recommender should consider:

- Fit with the collector profile
- Similarity to the three reference purchases

- Vintage or collector appeal
- Condition and ambiguity risk
- Seller/listing quality signals
- Whether the item feels distinctive enough to be worth surfacing

The output should be concise and useful: recommended watch, total cost, why it matches, and any risk notes.

EVALUATOR

The evaluator is a light agent or LLM check that reviews the top recommendations before they are shown.

It answers:

- Is the output faithful to the listing data?
- Does the reasoning match the collector profile?
- Did the recommendation accidentally ignore a risk signal?
- Are the budget and condition constraints clearly respected?
- Should the UI warn the user about anything?

The evaluator is not meant to become a large evaluation framework in the MVP. It is a quality gate for recommendation trust. Later, the system could support more formal evaluation such as precision, recall, F1, user feedback loops, and personalization from clicks or purchases, but that is outside the implemented scope.

UI

The web app is intentionally thin.

The assignment value is in the product reasoning and recommendation pipeline, not in a complex interface. The UI should let the user run the scout, review top recommendations, understand why each one matched, and see the budget and risk checks clearly.

MVP UI elements:

- Run watch search
- Top recommendations
- Why this matches
- Budget check: item price + shipping
- Risk flags
- Evaluator score or evaluator notes

TECH STACK

Backend: Python and FastAPI
Frontend: React and TypeScript

Database: PostgreSQL
LLM: OpenAI API

Data: normalized seed listings, collector profile, and normalized reference purchases

The backend is Python because it is the fastest and clearest way to build and explain the agent pipeline, validation logic, provider abstraction, and LLM calls during a code walkthrough. React and TypeScript provide a product-facing interface without overbuilding the frontend.

CONSTRAINTS

The app is local only. There is no public deployment requirement for this MVP.

The project should be easy to run, inspect, and explain in an interview walkthrough.

The implementation should favor correctness, traceability, and a presentable end-to-end flow over broad marketplace coverage or complex infrastructure.